

CONTRIBUTIONS.md

Overview

This document defines the recommended workflow for contributing changes to this repository. Even though branch protection is not enforced, contributors are expected to follow these guidelines to maintain stability, prevent bugs, and ensure clean project history.

Daily Workflow

1. **Pull the latest changes**

```
git checkout main
git pull origin main
git checkout -b xx-CODE/short-describe    xx: #issue CODE: Epic
git merge origin/main
```

2. **Work in small commits**

- Keep commits focused on one logical change at a time.
- Include meaningful messages:

```
git add .
git commit -m "CAM-ZOOM-03: Integrate mouse scroll input for zoom"
```

3. **Test locally before pushing**

- Run all tests and verify no regressions.
- If applicable, perform manual testing for UI/gameplay changes.

4. **Squash commits then Push to remote**

```
git rebase -i main
git push origin feature/camera-zoom
```

5. **Open a Pull Request (PR)**

- Even if working solo, open a PR and review your own code before merging.
- Use the PR description to summarize:
 - What was done - Why it was done
 - Any risks or side-effects

6. **Perform code review (self or team)**

- Code is consistent with project style, ensure:
 - No debug code or print statements
 - No commented-out blocks of old code
 - Descriptive commit messages

7. **Merge the PR into `main`**

- Once reviewed and tested:

```
git checkout main
git pull origin main
git merge feature/camera-zoom
git push origin main
```

8. **Delete the feature branch**

```
git branch -d feature/camera-zoom
git push origin --delete feature/camera-zoom
```

Best Practices

- Keep `main` stable and always deployable
- Treat PRs as checkpoints for code quality
- Link GitHub Issues or TODOs in commits or PRs
- Write tests for new features or bugs where possible

- Use `.gitignore` and never commit secrets or build artifacts
- Use semantic commit prefixes if helpful:
 - `feat:`, `fix:`, `chore:`, `test:`, `refactor:`

⚠ Risks to Avoid

Risk	Description
Direct commits to main	Can accidentally break builds or introduce unstable code
Large unreviewed merges	Makes debugging and reviewing harder later
Untracked debug/test code	May accidentally ship testing logic to production
Stale feature branches	Lead to merge conflicts and diverging histories
Lack of testing	Bugs sneak in due to skipped validation steps
Force pushes on main	Can cause data loss and rewrite shared history

🔧 Optional Enhancements

If you're working alone but want safety:

- Enable branch protection for `main` (manual merge only)
- Use GitHub Actions or shell scripts to run tests on PRs
- Create a `.pre-commit-config.yaml` for linting and formatting

Author: Mike Mattinson

Updated: Aug/14/2025 - r3

Updated: Aug/8/2025 - r2